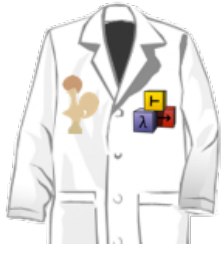
Forward Symbolic Execution for Trustworthy Automation of Binary Code Verification

Andreas Lindner[1], Karl Palmskog[2], Scott Constable[3], Mads Dam[2], Roberto Guanciale[2], and Hamed Nemati[2]

[1]Uppsala University, Uppsala, Sweden
[2]KTH Royal Institute of Technology, Stockholm, Sweden
[3]Intel Labs, Hillsboro, OR, United States



Unstructured Code: Hidden But Ever-Present

Unstructured code:

- jumps to labels, might have multiple exit points for statements
- often has control flow that is a "can of spaghetti"
- lacks structure such as typing information
- may leak information even when source code does not
- is still relevant as compiler output, hand-written assembly

RISC-V assembly indirect jump:

```
1:      beq     a1, zero, 4     # determine call
2:      la      t0, func1       # load address
3:      j       5               # jump to call
4:      la      t0, func2       # load address
5:      jalr    ra, t0, 0       # do call
```

"Depending on value in register a1, call either `func1` or `func2`"

Unstructured Code Needs Trustworthy Verification

Unstructured code verification tasks:

- functional correctness for translation validation
  - assembly produced by untrusted compilers
  - optimized assembly vs. unoptimized
- execution time bounds
  - real time systems
  - constant time cryptography
- low-level security properties
  - absence of side channels
  - microarchitectural constant time

Static Analysis Challenges for Unstructured Programs

- indirect jumps
- memory aliasing
- no reliable typing information
- many instruction set architectures (ISAs)
  - x86-64
  - ARMv7
  - ARMv8
  - RISC-V

Our Contributions

1. general symbolic execution theory, suitable for theorem provers
2. implementation of the theory in HOL4 theorem prover, instantiation for BIR intermediate language
3. implementation of automatic symbolic executors for BIR, applicable to RISC-V and Arm Cortex-M0 machine code
4. application and evaluation of our symbolic execution to prove
   - Hoare-style contracts for RISC-V
   - execution time bounds for Arm Cortex-M0

Symbolic Execution Ingredients

Concrete Side

- set of labels L
- concrete states s, a tuple consisting of
  - program counter $pc(s) = l$, where $l \in L$
  - $store(s) = \delta$, partially mapping variables to values
- total, deterministic transition step relation $s \rightarrow^n_L s'$

Symbolic Execution Ingredients

Concrete Side

- set of labels L
- concrete states s, a tuple consisting of
  - program counter $pc(s) = l$, where $l \in L$
  - $store(s) = \delta$, partially mapping variables to values
- total, deterministic transition step relation $s \rightarrow^n_L s'$

Symbolic Side

- symbolic state $\overline{s}$, a tuple consisting of
  - program counter $pc(\overline{s}) = l$, where $l \in L$
  - $store(\overline{s}) = \overline{\delta}$, mapping variables to symbolic expressions
  - path condition $path(\overline{s}) = \sigma$
- interpretation H, mapping symbols to concrete values

Symbolic Execution Example

```
Fun():
  a += 1
  if (b != 0):
    a += 2
  return
```

Initial symbolic state

$$\begin{aligned} \mathsf{a} &\mapsto \alpha \\ \mathsf{b} &\mapsto \beta \end{aligned}$$

Final symbolic states

σ: $\beta \neq 0$

$$\begin{aligned} \mathsf{a} &\mapsto \alpha + 3 \\ \mathsf{b} &\mapsto \beta \end{aligned}$$

σ: $\beta = 0$

$$\begin{aligned} \mathsf{a} &\mapsto \alpha + 1 \\ \mathsf{b} &\mapsto \beta \end{aligned}$$

Symbolic Execution Soundness

Definition (Matching state)

Th symbolic state $\overline{s}$ *matches* a concrete state s via the interpretation H, written $\overline{s} \overset{H}{\triangleright} s$, if $pc(\overline{s}) = pc(s)$, $H(store(\overline{s})) = store(s)$, and $H(path(\overline{s}))$.

Symbolic Execution Soundness

Definition (Matching state)

Th symbolic state $\overline{s}$ *matches* a concrete state s via the interpretation H, written $\overline{s} \overset{H}{\triangleright} s$, if $pc(\overline{s}) = pc(s)$, $H(store(\overline{s})) = store(s)$, and $H(path(\overline{s}))$.

Definition (Loosely matching state)

The symbolic state $\overline{s}'$ loosely matches the concrete state s' via H, $\overline{s}' \overset{H}{\blacktriangleright} s'$, if there is an interpretation $H' \supseteq H$ such that $\overline{s}' \overset{H'}{\triangleright} s'$.

Symbolic Execution Soundness

Definition (Matching state)

Th symbolic state $\overline{s}$ *matches* a concrete state s via the interpretation H, written $\overline{s} \overset{H}{\triangleright} s$, if $pc(\overline{s}) = pc(s)$, $H(store(\overline{s})) = store(s)$, and $H(path(\overline{s}))$.

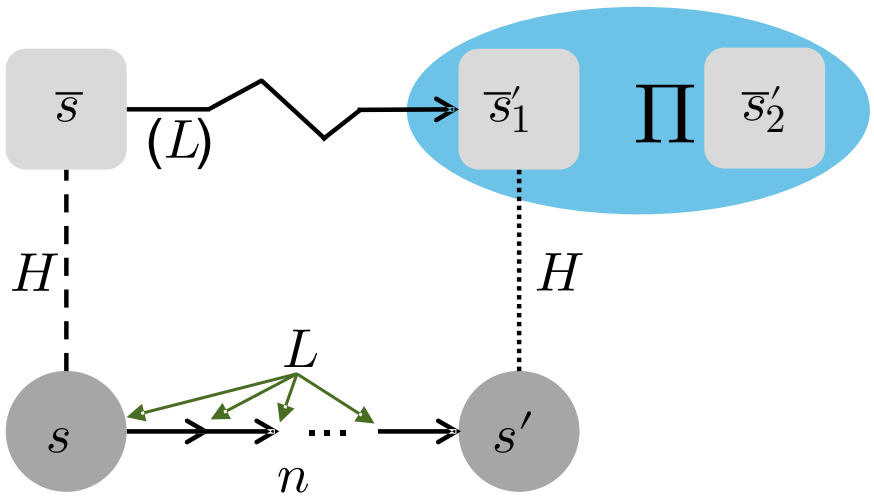
Definition (Loosely matching state)

The symbolic state $\overline{s}'$ loosely matches the concrete state s' via H, $\overline{s}' \overset{H}{\blacktriangleright} s'$, if there is an interpretation $H' \supseteq H$ such that $\overline{s}' \overset{H'}{\triangleright} s'$.

Definition (Symbolic execution soundness)

$\overline{s} \rightsquigarrow_L \Pi$ is a *progress structure* that is sound if for every interpretation $\downarrow_{\overline{s}} (H)$ and state $\overline{s} \overset{H}{\triangleright} s$, there exists $s \rightarrow_L^n s'$ where $n > 0$, and there exists $\overline{s}' \in \Pi$ such that $\overline{s}' \overset{H}{\blacktriangleright} s'$.

Symbolic Execution Soundness, Illustrated



$\vdash \boxed{\overline{s} \rightsquigarrow_L \Pi} \stackrel{\text{def}}{=}$ *for every interpretation* $\downarrow_{\overline{s}} (H)$ *and state* $\overline{s} \overset{\text{H}}{\triangleright} s$

exist $s \rightarrow_L^n s'$ *where* $n > 0$,

and $\overline{s}' \in \Pi$ *such that* $\overline{s}' \overset{\text{H}}{\blacktriangleright} s'$.

Symbolic Execution Theory Rules

Let $\mathit{symsem}(\overline{s})$ be the set of all symbolic successor states of $\overline{s}$, and let $\Lambda(\overline{s})$ be the set of symbols in $\overline{s}$.

$$\frac{\mathit{symsem}(\overline{s}) = \Pi \qquad \mathit{pc}(\overline{s}) \in L}{\vdash \overline{s} \rightsquigarrow_L \Pi} \text{ SYMBSTEP}$$

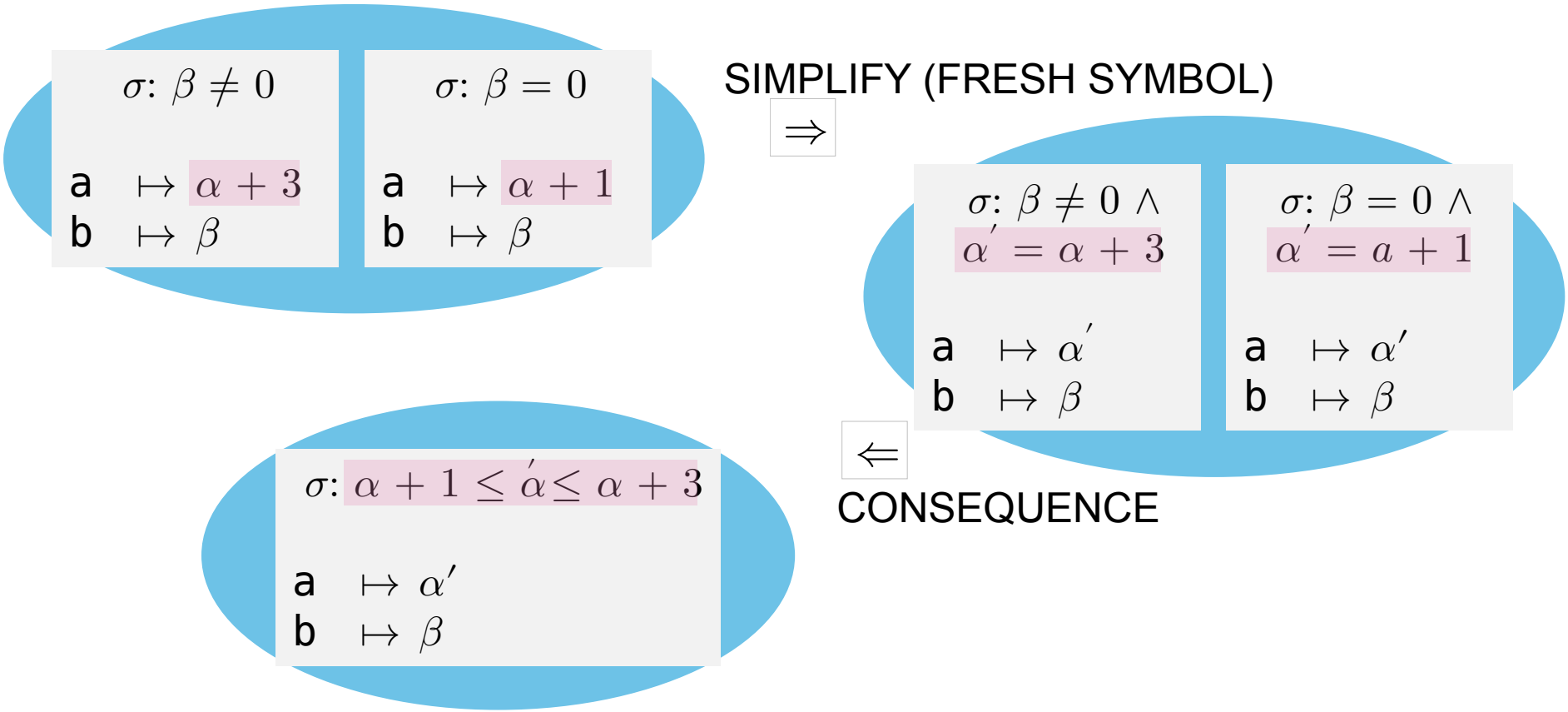
$$\frac{\vdash \overline{s} \rightsquigarrow_L \Pi \qquad \alpha \in \Lambda(\overline{s}) \qquad \Lambda(\overline{v}) \cap \mathit{fs}(\overline{s} \rightsquigarrow_L \Pi) = \emptyset}{\vdash \overline{s}[\overline{v}/\alpha] \rightsquigarrow_L \Pi[\overline{v}/\alpha]} \text{ SUBST}$$

Symbolic Execution Theory Rules, Continued

$$\frac{\vdash \overline{s} \rightsquigarrow_L \Pi \cup \{(l, \overline{\delta}, \phi)\} \qquad \overline{\delta}(x) = \overline{v} \qquad \forall H.\ H(\phi \wedge \alpha = \overline{v}'' \Rightarrow \overline{v} = \overline{v}') \qquad \alpha \notin \Lambda(\overline{s}) \cup \Lambda((l, \overline{\delta}, \phi)) \qquad \Lambda(\overline{v}'') \subseteq \Lambda((l, \overline{\delta}, \phi))}{\vdash \overline{s} \rightsquigarrow_L \Pi \cup \{(l, \overline{\delta}[x \mapsto \overline{v}'], \phi \wedge \alpha = \overline{v}'')\}}\ \textsc{simplify}$$

$$\frac{\overline{s}_1 \Rightarrow \overline{s}'_1 \qquad \overline{s}_2 \Rightarrow \overline{s}'_2 \qquad \vdash \overline{s}'_1 \rightsquigarrow_L \Pi \cup \{\overline{s}_2\} \qquad \Lambda(\overline{s}_1) \cap fs(\overline{s}'_1 \rightsquigarrow_L \Pi \cup \{\overline{s}_2\}) = \emptyset}{\vdash \overline{s}_1 \rightsquigarrow_L \Pi \cup \{\overline{s}'_2\}}\ \textsc{consequence}$$

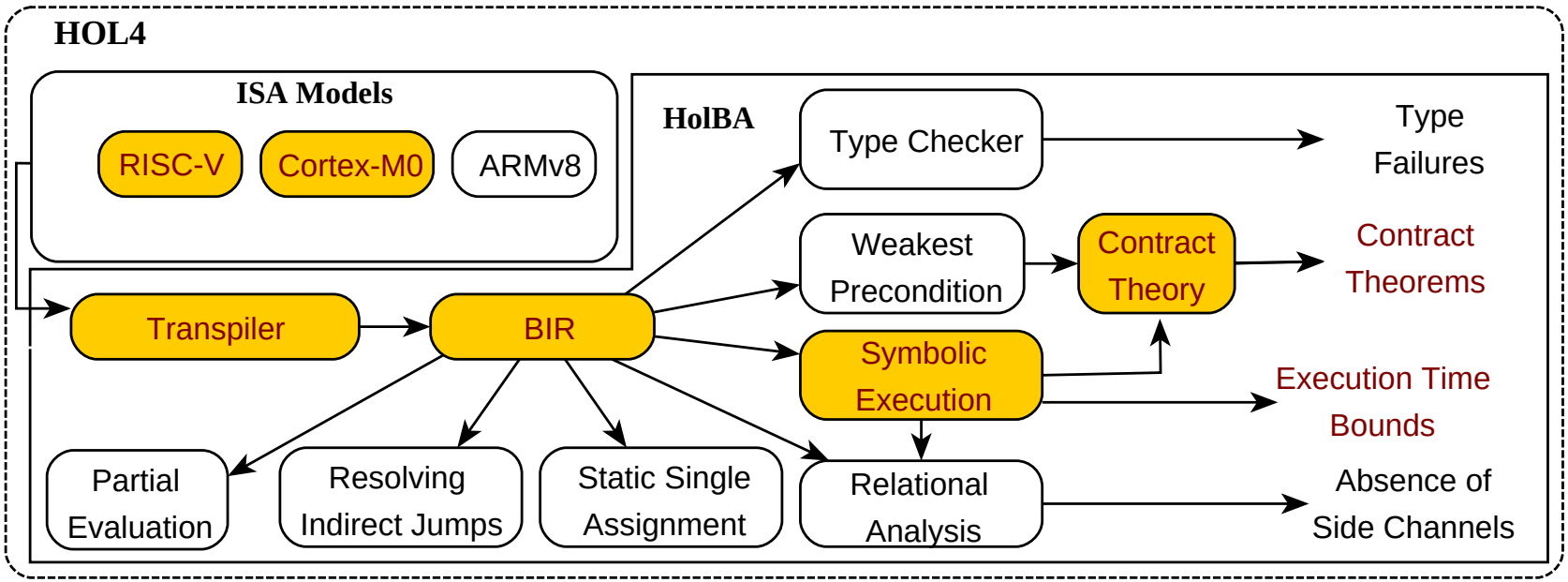
Symbolic Execution Interval Analysis



Implementation Approach

- Rules and their soundness formalized in HOL4 theorem prover
- Tooling implemented in Standard ML
- Standalone symbolic execution theory
- Instantiate in HolBA library
  - use BIR intermediate (abstract assembly) language
  - use existing translations from RISC-V / ARM to BIR
- Implement automatic symbolic executors for BIR

HolBA Architecture



Symbolic Executors and Progress Structures

- Symbolic execution rules are operations in abstract machine
- To use rules, they must be leveraged in **executors**
- Executors (for BIR) run and compose execution blocks and apply post-processing, building progress structures
- We designed executors for RISC-V and Cortex-M0
  - implemented in Standard ML
  - generate progress structure theorems
  - *proof producing* inside HOL4

Symbolic Execution Evaluation: Progress Structures

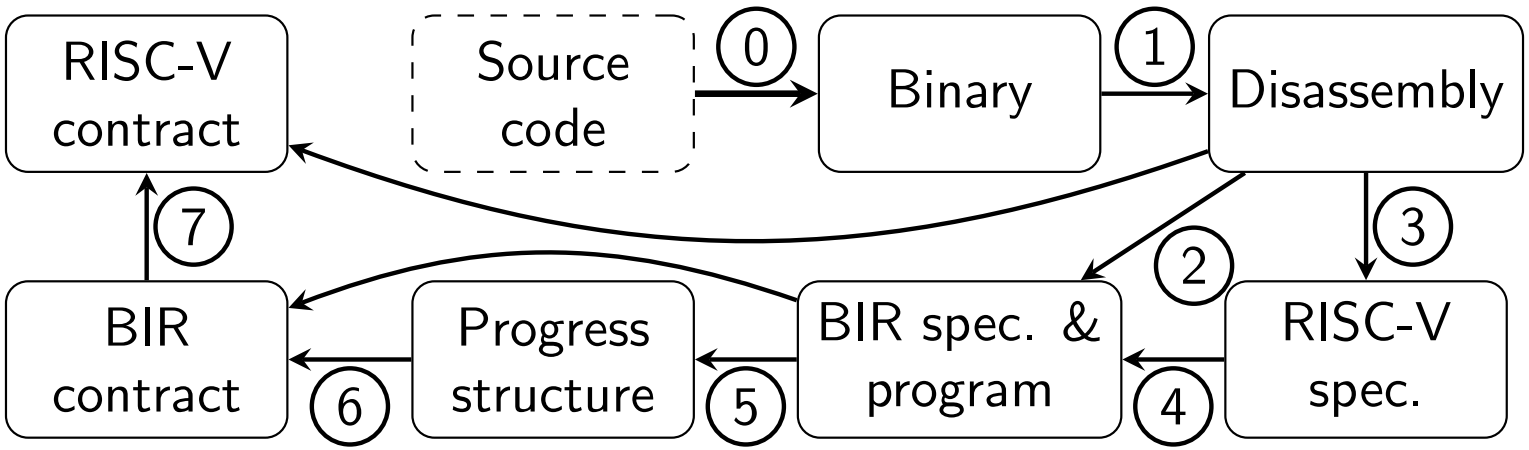
Program	**Description**	**#Instrs**	**Time**
aes-unopt	AES cipher, -O0 optimization	310	71.38 s
aes	AES cipher, -O1 optimization	130	6.84 s
chacha20	ChaCha20 cipher	177	46.28 s
incr	increment variable	1	0.04 s
isqrt	integer square root	6	0.12 s
kernel-trap	S3K kernel trap routines	79	9.21 s
mod2	modulo two computation	1	0.09 s
modexp	modular exponentiation	10	0.26 s
motor_set	nested calls and branching	66	3.94 s
swap	swap memory location	5	0.19 s

Application: Functional Correctness of RISC-V Binaries

- Leverage progress structures for Hoare-style binary contracts
- Build on earlier theory of binary contracts for RISC-V and BIR
- Start with RISC-V contract, translate to BIR contract
- Use precondition as path condition in BIR program, try to prove strongest postcondition
- Translate BIR contract to RISC-V level
- Connection between BIR and RISC-V maintained by simulation

RISC-V Contract Verification Workflow

0. Compilation 1. Disassembly 2. Lifting 3. RISC-V specification



4. BIR specification 5. Symbolic execution 6. Symbolic transfer

7. Backlifting

Progress Structures for Contract Proofs

Theorem

Given $\vdash \overline{s} \rightsquigarrow_L \Pi$, *let* $\bar{L}$ *be the complement of* L. *Then, if*

- *for every* s, *if* $pc(s) = pc(\overline{s})$ *and* $P(s)$, *then there exists* H *such that* $\overline{s} \overset{H}{\triangleright} s$
- *for every* $\overline{s} \overset{H}{\triangleright} s$, $\overline{s}' \in \Pi$, *and* $\overline{s}' \overset{H}{\blacktriangleright} s'$ *if* $P(s)$ *then* $Q(s')$
- *for every* $\overline{s}' \in \Pi$, $pc(\overline{s}') \in \bar{L}$ *holds,*

we have that for every state such that $pc(s) = l$ *and* $P(s)$ *there is a state* s' *and an* $n > 0$ *such that* $s \rightarrow_L^n s'$, $pc(s') \in \bar{L}$, *and* $Q(s')$.

RISC-V Contract Verification Example

```
#include <stdint.h>
uint64_t incr(uint64_t i) { return i + 1; }
```

```
incr:     file format elf64-littleriscv
Disassembly of section .text:
0000000000010488 <incr>:
   10488:       00150513                addi   a0,a0,1
   1048c:       00008067                ret
```

RISC-V Contract Verification Example, Continued

```
incr:     file format elf64-littleriscv
Disassembly of section .text:
0000000000010488 <incr>:
   10488:       00150513                addi   a0,a0,1
   1048c:       00008067                ret
```

```
Definition riscv_incr_pre_def:
 riscv_incr_pre (pre_x10:word64) (m:riscv_state) =
 (m.c_gpr m.procID 10w = pre_x10)
End
Definition riscv_incr_post_def:
 riscv_incr_post (pre_x10:word64) (m:riscv_state) =
 (m.c_gpr m.procID 10w = pre_x10 + 1w)
End
```

RISC-V Contract Verification Example, Continued

```
incr:     file format elf64-littleriscv
Disassembly of section .text:
0000000000010488 <incr>:
   10488:       00150513                addi   a0,a0,1
   1048c:       00008067                ret
```

```
Theorem riscv_cont_incr:
 riscv_cont incr_progbin 0x10488w {0x1048cw}
  (riscv_incr_pre pre_x10) (riscv_incr_post pre_x10)
```

"When the binary program is executed from label 10488 using a RISC-V machine in a state satisfying the precondition which reaches label 1048c, the postcondition will hold."

Case Studies for RISC-V Functional Verification

- ChaCha20 cipher
  - compiled -O1 from reference implementation
  - verify encryption loop body (XORs)
  - verification takes $\sim$ 4 minutes on modern machine
- Separation kernel (S3K) trap routines
  - handwritten RISC-V assembly
  - uses special RISC-V registers
  - not possible to verify without binary analysis
  - verification takes $\sim$ 7 minutes on modern machine

Example: ChaCha20 Translation Validation

```
chacha_line a b c d s (m : word32 -> word32) =
 let m = (a =+ (m a) + (m b)) m in
 let m = (d =+ ((m d) XOR (m a)) #<<~ s) m in m
```

```
108c8:   00fa0a3b              addw   s4,s4,a5
108cc:   01454533              xor    a0,a0,s4
108d0:   00851b1b              slliw  s6,a0,0x8
108d4:   0185551b              srliw  a0,a0,0x18
108d8:   00ab6b33              or     s6,s6,a0
```

```
Theorem replace_word_rol_bv_or_shifts:
 !x s : word32. s <=+ 31w ==>
  x #<<~ s =
  (x <<~ s) || (x >>>~ (32w - s))
```

Application: Execution Time Bounds for Cortex-M0

- Arm Cortex-M0 processor has short pipeline and no caches
- without complex pipeline and speculative execution, instruction time becomes predictable
- ALU instructions take 1 cycle
- memory interactions take 1 extra cycle
- control flow changes take 2 extra cycles
- L3 model of Arm Cortex-M0 maintains clock cycle counter

Execution Time Representation and BCET/WCET

- In BIR, Cortex-M0 programs increment cycle counter c
- Whan symbolic paths merge, c is approximated in an interval
- $[\alpha_{\mathsf{c}} + 5, \alpha_{\mathsf{c}} + 8]$ consists of $\mathsf{c} \mapsto \alpha'$ in the store and the path condition conjunct $\alpha_{\mathsf{c}} + 5 \leq \alpha' \leq \alpha_{\mathsf{c}} + 8$
- left bound $(\alpha_{\mathsf{c}} + 5)$ corresponds to best-case time (BCET)
- right bound $(\alpha_{\mathsf{c}} + 8)$ corresponds to worst-case time (WCET)
- analysis is sound with respect to L3 semantics of Cortex-M0
- we compared to testing on hardware and analysis of aiT tool

Evaluation of Execution Time Bounds Verification

Program		Testing		aiT	HolBA-SE		
name	**#instrs**	**BCET**	**WCET**	**WCET**	**BCET**	**WCET**	**eval**
ldldbr8	34	60	60	55	60	60	1 s
aes	543	3761	3761	3761	3761	3761	8.4 min
modexp	130	54358	93540	197721	43704	197560	11.4 min
motor_set	165	272	280	283	264	280	49 s
motor_set o3	112	73	91	92	68	96	89 s
pid	2257	4589	5483	7521	1849	7529	61.8 min

Comparison to Binary Analysis Frameworks

- HolBA follows design of binary analysis frameworks ...
- ... but provides trustworthy proof automation
- binary contracts depend on:
  - L3 model of ISAs
  - HOL4 proof checker
  - Z3 for discharging (BIR) conditions, for performance
- symbolic execution is deeply embedded in theorem prover
  - makes reasoning possible about theory itself in HOL4
  - enables reusability outside BIR

Conclusions

- general theory of symbolic execution for unstructured code
- implemented in HOL4, instantiated for BIR
- machine-checked proofs of contracts and execution time
- open source (BSD-3-Clause)
- integrated into HolBA on GitHub

`https://github.com/kth-step/holba`

Automation of Binary Slicing and Modular Verification

- loops not handled automatically
- symbolic execution with improper boundaries diverges
- symbolic state can explode
- reason at contract level about “pieces”

Automated Side Channel Analysis

- prove absence of trace-driven/timing side channels
- focus on ARMv8
- symbolic execution and contract backlifting works
- add observations from side channel models to BIR program